

POCKETSMART AI

Project Topic Links

Reference Links for Smart Budget & Recommendation Assistant

1. Project Overview

This document provides the main topic links relevant to the PocketSmart AI project. The supplied project describes a GenAI-powered recommendation system for Home Interior, Party Planning and Jewelry recommendations, using HTML/CSS/JavaScript on the frontend, FastAPI/Flask-related backend components, Gemini AI, and third-party platforms.

2. Project Topics and Links

S.No	Topic	Purpose in Project	Official Link
1	PocketSmart AI Project	Main project concept: budget-aware recommendations for home, party and jewelry planning.	Project document
2	Home Interior Planning	Home decor, furniture, lighting and room-based recommendations.	IKEA / Amazon
3	Party Planning	Event budget, guests, catering, venue and decoration recommendations.	Swiggy / Zomato / OYO
4	Jewelry Recommendations	Occasion, style, budget and optional outfit-image based recommendations.	Amazon / Flipkart
5	Gemini API	AI processing, structured recommendations and multimodal input handling.	https://ai.google.dev/api
6	FastAPI	Backend API routes and frontend-backend integration.	https://fastapi.tiangolo.com/
7	Python	Backend programming and API development.	https://www.python.org/
8	Amazon	Product sourcing/reference platform mentioned for home and jewelry recommendations.	https://www.amazon.in/
9	Flipkart	Product sourcing/reference	https://www.flipkart.com/

		platform mentioned for jewelry and other products.	
10	IKEA India	Home furniture, decor and interior product reference platform.	https://www.ikea.com/in/en/
11	Swiggy	Food/catering reference platform for party planning.	https://www.swiggy.com/
12	Zomato	Food/catering reference platform for party planning.	https://www.zomato.com/
13	OYO	Accommodation/venue-related reference platform for event planning.	https://www.oyorooms.com/

3. Development Documentation Links

Gemini API Reference

<https://ai.google.dev/api>

Official Gemini API reference for API capabilities and endpoints.

Gemini API Getting Started

<https://ai.google.dev/gemini-api/docs/get-started>

Official guide for getting started with Gemini API.

FastAPI Documentation

<https://fastapi.tiangolo.com/>

Official FastAPI documentation.

FastAPI Tutorial

<https://fastapi.tiangolo.com/tutorial/>

Step-by-step FastAPI tutorial.

Python Official Website

<https://www.python.org/>

Python language, downloads and documentation.

4. Frontend Topics

- HTML Frontend Interface
- Responsive HTML Forms
- Home Interior Planner
- Party Planner
- Jewelry Planner
- Dynamic Recommendation Display using JavaScript
- Recommendation Card Layout
- Login and Register Pages
- User Dashboard

- Recommendation History
- Jinja2 Templates

5. Backend and API Topics

- /generate-home – Home interior recommendation route
- /generate-party – Party recommendation route
- /generate-jewelry – Jewelry recommendation route
- /register – User registration
- /login – User authentication
- /logout – Session termination
- /token – Token generation
- /session-info – Session information
- /session-data – Session-specific data
- /history – Recommendation history
- /recommendations-details – Detailed AI recommendation results

6. Source Alignment

The supplied project specifically identifies Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO as example platforms for product or service sourcing, and identifies HTML/CSS + JavaScript as the frontend technology. It also describes separate Home, Party and Jewelry planner modules.

Note: The URLs in this document are official/reference websites. The supplied project document mentions third-party platforms as data sources, but it does not establish that live API access or direct integration with each platform is currently implemented.

7. Conclusion

These topic links can be used as a reference section in the PocketSmart AI project report. They cover the AI technology, backend framework, programming language, frontend development topics, and the external platforms described in the project.

POCKETSMART AI

Define FastAPI Routes

Project Documentation

1. Introduction

PocketSmart AI uses a FastAPI-powered backend to connect the frontend interface with the Gemini-based AI recommendation engine. FastAPI routes are used to receive user inputs, process planner requests, manage authentication, and return the required recommendation or session information.

2. Objective

The objective of defining FastAPI routes is to create clear backend endpoints for the Home Planner, Party Planner, Jewelry Planner, user authentication, session management, and recommendation details. These routes allow the frontend to communicate with the backend in an organized manner.

3. FastAPI Route Architecture

The FastAPI backend acts as the bridge between the frontend interface and the Gemini AI recommendation engine. The routes are organized according to the functions provided by PocketSmart AI, making the backend modular and maintainable.

4. Main Planner Routes

Route	Module	Purpose
/generate-home	Home Planner	Accepts home-related preferences and budget information and returns tailored product suggestions.
/generate-party	Party Planner	Processes party details and guest count to recommend venue, food, and decoration options.
/generate-jewelry	Jewelry Planner	Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

5. User Authentication Routes

Route	Function
/register	Handles new user registration by accepting and securely storing user credentials.
/login	Authenticates existing user credentials and starts a user session after successful login.
/logout	Terminates the current user session and redirects the user to the login page.

/token	Issues a secure JWT token after successful authentication for authorized access to protected API routes.
--------	--

6. Session and Recommendation Routes

Route	Purpose
/session-info	Retrieves metadata about the current user session, such as user ID and login status.
/session-data	Returns detailed session-specific data used for personalization and recommendation tracking.
/recommendations-details	Returns detailed AI-generated product recommendations based on budget, preferences, and selected category.
/history	Retrieves the user's past recommendation queries and results for review or reuse.

7. Route Processing Flow

1. The user enters budget and preference details through the frontend.
2. The frontend sends the request to the appropriate FastAPI route.
3. The FastAPI backend receives and processes the request.
4. The relevant planner route communicates with the recommendation logic and Gemini AI layer.
5. The generated recommendation is returned to the frontend for display.
6. Authentication and session routes manage user access and personalized session information.

8. Example Route Structure

The project documentation defines routes such as /generate-home, /generate-party, and /generate-jewelry for the three main planners. Authentication routes include /register, /login, /logout, and /token. Session routes include /session-info and /session-data.

9. Benefits of FastAPI Routes

- Provides organized communication between the frontend and backend.
- Separates Home, Party, and Jewelry planner functionality.
- Supports user authentication and session control.
- Allows AI-generated recommendations to be requested through dedicated endpoints.
- Makes the backend modular and easier to maintain.

10. Expected Result

After defining the FastAPI routes, PocketSmart AI should be able to receive requests for its different planner modules, process authenticated user sessions, and return AI-generated recommendations and session-related information through structured backend endpoints.

11. Conclusion

Defining FastAPI routes is an important part of the PocketSmart AI backend. The routes connect the frontend with the AI recommendation system and provide separate endpoints for Home, Party, and Jewelry planning. Authentication, session management, recommendation details, and history routes further support a personalized and organized user experience.

Source basis: PocketSmart AI project documentation – Milestone 3: Backend – FastAPI Integration, Activity 3.1.

MODULAR ARCHITECTURE SETUP

PocketSmart AI – FastAPI Backend Architecture & Integration Guide

Project	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Milestone	Milestone 3 – Backend: FastAPI Integration (main.py)
Activity	Activity 3.2 – Modular Architecture Setup
Backend	FastAPI-powered backend
AI Integration	Gemini-based recommendation engine
Architecture Goal	Clean, maintainable, modular API and service organization

Source basis: This document is prepared from the uploaded PocketSmart AI project documentation. It focuses specifically on the project's Modular Architecture Setup activity and its stated FastAPI, CORS, session, routing, and structured input/output responsibilities.

Prepared for project implementation, demonstration, documentation, and review.

1. Modular Architecture Setup - Overview

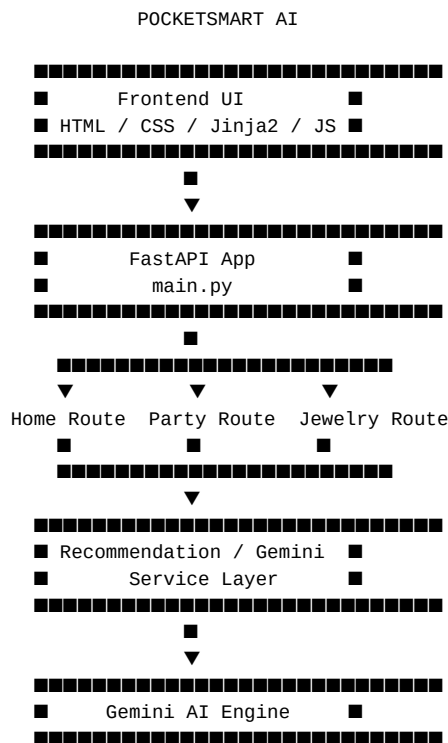
The Modular Architecture Setup is part of the PocketSmart AI FastAPI backend milestone. The backend acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in the project. The architecture is intended to provide clean and maintainable API endpoints for the Home, Party, and Jewelry domains.

The project documentation states that this backend also includes CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models for reliable frontend rendering.

2. Main Objectives

- Create a modular FastAPI backend for the PocketSmart AI application.
- Connect frontend requests with the Gemini-based recommendation engine.
- Keep Home, Party, and Jewelry planner endpoints clearly organized.
- Support CORS for frontend-to-backend communication.
- Support session handling for personalized user experiences.
- Use structured input/output models for consistent API communication.
- Provide a clean foundation for startup, routing, authentication, and recommendation services.

3. High-Level Architecture



Supporting layers:

- CORS
- Authentication
- Session handling
- Input / Output models
- History / recommendation details

4. FastAPI Application Entry Point

The main application file provides the central entry point for the backend. It is responsible for creating the FastAPI application, registering routers, configuring middleware such as CORS, and starting the application through the project's startup process.

```
# main.py - simplified architecture example

from fastapi import FastAPI
from routes.home import router as home_router
from routes.party import router as party_router
from routes.jewelry import router as jewelry_router

app = FastAPI(title="PocketSmart AI")

app.include_router(home_router)
app.include_router(party_router)
app.include_router(jewelry_router)

@app.get("/")
async def root():
    return {"message": "PocketSmart AI backend is running"}
```

The exact source project contains a startup function and a `__main__` entry point that starts the FastAPI application using Uvicorn.

5. Route Organization

Route	Purpose
/generate-home	Accepts home-related preferences and budget to return tailored product suggestions.
/generate-party	Processes party details and guest count to recommend venue, food, and decoration items.
/generate-jewelry	Uses text and optional image inputs to suggest jewelry based on outfit and occasion.
/recommendations-details	Returns detailed AI-generated recommendations based on budget, preferences, and category.
/history	Retrieves previous recommendation queries and results.

These endpoints and their responsibilities are taken from the uploaded project's FastAPI milestone.

`filecite\turn0file0\L193-L209`

6. Structured Input and Output Models

Structured models provide a defined format for data entering and leaving the backend. This supports consistent communication between the frontend, FastAPI routes, recommendation logic, and Gemini layer.

```
# Example structure

class HomeRequest(BaseModel):
    budget: float
    room_type: str
    preferences: str

class PartyRequest(BaseModel):
    budget: float
    guest_count: int
    event_type: str

class JewelryRequest(BaseModel):
    budget: float
    occasion: str
    style: str
    image_path: str | None = None
```

These examples illustrate the architectural role of structured models. They are not presented as the exact source code from the uploaded project.

7. CORS Configuration

CORS (Cross-Origin Resource Sharing) allows the frontend and backend to communicate when they are served from different origins. In PocketSmart AI, CORS is part of the backend integration so that frontend planner requests can reach the FastAPI endpoints.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

For deployment, the allowed origins should be restricted to the application's trusted frontend origins.

8. Session Handling

Session handling supports personalization and user-specific recommendation data. The project includes session-info and session-data functionality as part of its backend architecture.

Endpoint	Purpose
/session-info	Retrieves metadata about the current user session, such as user ID and login status.
/session-data	Returns detailed session-specific data used for personalization and recommendation tracking.

9. Startup and Main Function

The source project specifies a startup function that initializes essential application services and loads configuration settings when the FastAPI server starts. The `__main__` entry point starts the server using Uvicorn when the script is executed directly.

```
@app.on_event("startup")
async def startup():
```

```

# Initialize application services
# Load required configuration
pass

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

10. Complete Request Flow

1. User submits planner data in the frontend.
↓
2. Frontend sends request to FastAPI.
↓
3. FastAPI route receives and validates the request.
↓
4. Route calls recommendation logic.
↓
5. Recommendation logic prepares Gemini input.
↓
6. Gemini processes the request.
 - Text input
 - Optional image input for Jewelry
 ↓
7. Backend formats the AI result.
↓
8. FastAPI returns structured output.
↓
9. Frontend displays recommendations.

11. Benefits of the Architecture

Area	Benefit
Maintainability	Each backend responsibility can be modified in its appropriate layer.
Scalability	New planner modules can be added without redesigning the entire application.
Testing	Routes, services, models, and authentication can be tested separately.
Reusability	Shared Gemini/recommendation utilities can support multiple planners.
Frontend integration	Structured API responses simplify rendering and UI updates.
Personalization	Session functionality can be maintained independently of AI recommendation logic.

12. Modular Architecture Validation Checklist

- ☐ FastAPI application starts successfully.
- ☐ Home, Party, and Jewelry routers are registered.
- ☐ Structured request/response models are used.
- ☐ CORS configuration permits the intended frontend origin.
- ☐ Session information can be retrieved.
- ☐ Session-specific data can be retrieved.
- ☐ Recommendation-details endpoint returns structured data.
- ☐ History endpoint retrieves previous recommendation data.
- ☐ Startup function initializes required services/configuration.
- ☐ Uvicorn can start the FastAPI application.

13. Alignment with the Project Document

The uploaded project describes Milestone 3 as the FastAPI-powered backend acting as the bridge between the frontend and Gemini recommendation engine. It explicitly lists modular routes, CORS, session handling, structured input/output models, recommendation details, history, startup, and Uvicorn execution.

Source reference: `filecite[turn0file0]L185-L217`

14. Conclusion

The Modular Architecture Setup provides the structural foundation for PocketSmart AI's FastAPI backend. By separating planner routes, recommendation processing, structured data, authentication/session features, CORS configuration, and startup behavior, the application can maintain a clear communication path between the frontend and Gemini-powered recommendation engine.

Project demonstration recommendation: Show the FastAPI application running, demonstrate the planner routes, verify session endpoints, display a structured recommendation response, and explain how each component belongs to a specific architectural responsibility.

POCKETSMART AI

CORS & Static Routing

Project Documentation

1. Introduction

PocketSmart AI uses a FastAPI-powered backend to connect the frontend interface with the Gemini-based AI recommendation engine. CORS and static routing are part of the backend configuration that supports communication between the frontend and backend and helps the application serve its required resources.

2. Objective

The objective of configuring CORS and static routing is to support smooth communication between the PocketSmart AI frontend and FastAPI backend while maintaining an organized routing structure for the application.

3. CORS Configuration

CORS stands for Cross-Origin Resource Sharing. In PocketSmart AI, CORS configuration allows the frontend interface to communicate with the FastAPI backend when they operate across different origins. This is important for sending planner requests and receiving AI-generated results through backend API endpoints.

4. Role of CORS in PocketSmart AI

- Allows frontend-to-backend communication.
- Supports requests from the application interface to FastAPI API endpoints.
- Helps the frontend access responses returned by the backend.
- Supports the integration between the user interface and the AI recommendation services.

5. Static Routing

Static routing refers to the organized handling and serving of the application's static resources and frontend routes. In PocketSmart AI, the frontend is built using HTML, CSS, and Jinja2 templates, while FastAPI provides the backend routing structure. The routing setup helps connect the application's pages and resources with the backend.

6. Important Application Routes

Route / Resource	Purpose
/generate-home	Handles Home Planner requests.
/generate-party	Handles Party Planner requests.
/generate-jewelry	Handles Jewelry Planner requests.
/login	Handles user authentication.

/register	Handles new user registration.
/logout	Terminates the current user session.
Frontend static resources	Supports the HTML/CSS/Jinja2-based user interface.

7. CORS and Routing Workflow

1. The user interacts with the PocketSmart AI frontend.
2. The frontend sends a request to the required FastAPI endpoint.
3. CORS configuration permits the required cross-origin communication.
4. FastAPI receives and processes the request.
5. The backend communicates with the relevant recommendation and AI logic.
6. The generated response is returned to the frontend and displayed to the user.

8. Benefits

- Provides smooth communication between frontend and backend.
- Supports the planner API endpoints used by PocketSmart AI.
- Keeps the backend routing structure organized.
- Helps the frontend receive AI-generated recommendation results.
- Supports a clean and maintainable application architecture.

9. Expected Result

After configuring CORS and static routing, the PocketSmart AI frontend should be able to communicate with the FastAPI backend through the defined routes. The application can then process planner requests and return the required results to the user interface.

10. Conclusion

CORS and static routing are important parts of the PocketSmart AI backend architecture. CORS supports communication between the frontend and backend, while the routing structure organizes the application's endpoints and frontend resources. Together, they help provide smooth interaction between the user interface, FastAPI backend, and AI recommendation system.

Source basis: PocketSmart AI project documentation – Milestone 3: Backend – FastAPI Integration, Activity 3.3.

PocketSmart AI

Startup and Main Function

Application Entry Point — main.py

Name	Divya Dharshini S
Project Title	PocketSmart AI
Component	Application Startup / Entry Point
Technology Stack	Python 3, FastAPI, Uvicorn

1. Technical Overview of Application Initialisation

The main.py module is the single entry point of the PocketSmart AI backend. Its responsibility is limited but critical: instantiate the FastAPI application object, attach cross-cutting middleware (such as CORS), mount static assets for the frontend, register the feature-specific routers (including the budget-analysis routes described in the previous section), expose a minimal root endpoint for health/status checks, and finally start the Uvicorn ASGI server when the file is executed directly.

Separating startup concerns into their own module — rather than mixing them into the business-logic routers — keeps the application composable. Additional routers, middleware, or startup/shutdown event handlers can be added to main.py in the future (for example, a database connection pool) without touching the budget-calculation logic itself.

CORS middleware is enabled to allow the HTML/JavaScript frontend, which issues asynchronous fetch() calls to the API, to communicate with the backend even when served from a different origin during local development. In a production deployment, allow_origins would be restricted to the specific deployed frontend domain rather than left as a wildcard.

2. Entry Point Code (main.py)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
import uvicorn

from routers.budget import router as budget_router

# -----
# Application initialisation
# -----

app = FastAPI(
    title="PocketSmart AI",
    description="An AI-powered budget tracker using FastAPI and the "
                "Google Gemini API – Naan Mudhalvan Project Submission.",
    version="1.0.0",
)

# Allow the HTML/JS frontend (served separately or from /static) to
```

```

# call this API during development without CORS being blocked
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Serve the static frontend (index.html, CSS, JS) directly from FastAPI
app.mount("/static", StaticFiles(directory="static"), name="static")

# Register the budget-related routes defined in routers/budget.py
app.include_router(budget_router)

# -----
# Root endpoint – simple landing / status route
# -----

@app.get("/")
async def root():
    """
    Root endpoint of the PocketSmart AI backend.
    Confirms the server is running and points to key resources.
    """
    return {
        "message": "PocketSmart AI backend is running.",
        "docs": "/docs",
        "frontend": "/static/index.html",
    }

# -----
# Server execution configuration
# -----

if __name__ == "__main__":
    # Launches the Uvicorn ASGI server when main.py is run directly
    # (i.e. "python main.py"), rather than via an external process
    # manager. reload=True is convenient for development and should
    # be disabled in a production deployment.
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8000,
        reload=True,
    )

```

3. Execution Instructions

- Development: run “python main.py” — Uvicorn starts with auto-reload enabled on port 8000.
 - Production: run “uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4” via a process manager, with reload disabled.
 - Interactive API documentation is auto-generated by FastAPI and available at /docs (Swagger UI) and /redoc.
-

POCKETSMART AI

HTML Frontend Interface – Project Document

Smart Budget & Recommendation Assistant

1. Introduction

PocketSmart AI is a budget-aware recommendation platform designed to help users plan home interiors, parties, and jewelry purchases. The project documentation specifies a lightweight, responsive frontend using HTML, CSS and JavaScript/Jinja2 templates. The frontend collects budgets, preferences and optional images, submits planner inputs to backend routes, and displays AI-generated recommendations in structured card layouts.

2. Objective of the HTML Frontend

- Provide a clean and responsive user interface for PocketSmart AI.
- Allow users to select Home, Party, or Jewelry planning.
- Collect budget and category-specific inputs through interactive forms.
- Support optional image upload for the Jewelry Planner.
- Send planner requests to the corresponding backend POST endpoints.
- Display AI-generated recommendations as readable product/service cards.
- Provide authentication, dashboard, history, and navigation pages.

3. Frontend Technology Stack

Technology	Purpose
HTML5	Page structure, forms, navigation and content
CSS3	Responsive layout, cards, buttons, spacing and visual styling
JavaScript	Dynamic form behavior, validation and recommendation rendering
Jinja2	Server-side templates and dynamic values when used with the backend
FastAPI	Backend routes consumed by the frontend
Gemini 1.5 Flash Pro	AI-powered budget interpretation and recommendation generation

4. Frontend Page Structure

Home Page: Introduces PocketSmart AI and provides navigation to the planner modules.

Register Page: Allows new users to create an account.

Login Page: Authenticates existing users.

User Dashboard: Shows recent recommendations, saved queries and personalized information.

Home Interior Planner: Collects home budget, room details and required quantities.

Home Recommendations: Displays AI-generated home product suggestions.

Party Planner: Collects event type, budget, guest count and venue details.

Party Recommendations: Displays food, venue and decoration recommendations.

Jewelry Planner: Collects budget, occasion, style and optional outfit image.

Jewelry Recommendations: Displays personalized jewelry recommendations.

Recommendation History: Shows previous queries and recommendation results.

5. Main Interface Design

The home interface should use a simple navigation bar, a clear hero section, planner category cards, and a responsive layout. Each planner card can contain a short description and a call-to-action button. The planner forms should use labels, input controls, validation messages and a prominent Generate Recommendations button.

Recommendation results should be presented in card-like layouts containing the product or service name, platform, estimated price, category, relevant details and an action/link area.

6. HTML Frontend – Main Page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PocketSmart AI</title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <header class="navbar">
    <div class="logo">PocketSmart AI</div>
    <nav>
      <a href="/">Home</a>
      <a href="/dashboard">Dashboard</a>
      <a href="/history">History</a>
      <a href="/login">Login</a>
    </nav>
  </header>

  <main>
    <section class="hero">
      <h1>Smart Budget. Smarter Recommendations.</h1>
      <p>Plan your home, party or jewelry purchase within your budget.</p>
    </section>

    <section class="planner-grid">
      <article class="planner-card">
        <h2>Home Interior</h2>
        <p>Plan rooms, furniture, lighting and decor.</p>
```

```

    <a class="btn" href="/home-planner">Start Planner</a>
</article>

<article class="planner-card">
  <h2>Party Planner</h2>
  <p>Plan food, venue and decoration for your event.</p>
  <a class="btn" href="/party-planner">Start Planner</a>
</article>

<article class="planner-card">
  <h2>Jewelry Planner</h2>
  <p>Find occasion-based jewelry within your budget.</p>
  <a class="btn" href="/jewelry-planner">Start Planner</a>
</article>
</section>
</main>

<footer>
  <p>&copy; 2026 PocketSmart AI</p>
</footer>
</body>
</html>

```

7. Home Interior Planner Form

```

<form action="/generate-home" method="POST" class="planner-form">
  <label for="home_budget">Total Budget</label>
  <input id="home_budget" name="budget" type="number"
    min="1" required placeholder="Enter budget">

  <label for="room">Room Type</label>
  <select id="room" name="room" required>
    <option value="">Select room</option>
    <option>Living Room</option>
    <option>Kitchen</option>
    <option>Bedroom</option>
  </select>

  <label for="items">Items / Quantities</label>
  <textarea id="items" name="items"
    placeholder="Example: 4 lights, 2 fans, 1 dining table"></textarea>

  <button type="submit">Generate Recommendations</button>
</form>

```

8. Party Planner Form

```

<form action="/generate-party" method="POST" class="planner-form">
  <label>Total Budget</label>
  <input name="budget" type="number" min="1" required>

  <label>Number of Guests</label>

```

```

<input name="guests" type="number" min="1" required>

<label>Event Type</label>
<select name="event_type" required>
  <option>Birthday</option>
  <option>Corporate</option>
  <option>Wedding</option>
</select>

<label>Venue Details</label>
<input name="venue" type="text" placeholder="Preferred location or venue">

<button type="submit">Plan My Party</button>
</form>

```

9. Jewelry Planner Form

```

<form action="/generate-jewelry" method="POST"
  enctype="multipart/form-data" class="planner-form">
  <label>Budget</label>
  <input name="budget" type="number" min="1" required>

  <label>Occasion</label>
  <select name="occasion" required>
    <option>Wedding</option>
    <option>Party</option>
    <option>Festival</option>
    <option>Casual</option>
  </select>

  <label>Style Preference</label>
  <input name="style" type="text"
    placeholder="Traditional, modern, elegant...">

  <label>Outfit Image (Optional)</label>
  <input name="outfit_image" type="file" accept="image/*">

  <button type="submit">Find Jewelry</button>
</form>

```

10. CSS Styling

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f5f7fb;
  color: #1f2937;
}

.navbar {
  display: flex;
  justify-content: space-between;

```

```
align-items: center;
padding: 16px 6%;
background: #ffffff;
box-shadow: 0 2px 10px rgba(0,0,0,.08);
}
```

```
.navbar nav a {
margin-left: 18px;
text-decoration: none;
color: #374151;
}
```

```
.hero {
text-align: center;
padding: 70px 20px 40px;
}
```

```
.planner-grid {
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 24px;
padding: 30px 6%;
}
```

```
.planner-card {
background: #ffffff;
padding: 28px;
border-radius: 14px;
box-shadow: 0 4px 18px rgba(0,0,0,.08);
}
```

```
.btn, button {
display: inline-block;
padding: 11px 18px;
border: 0;
border-radius: 8px;
cursor: pointer;
text-decoration: none;
background: #2563eb;
color: white;
}
```

```
.planner-form {
max-width: 650px;
margin: 30px auto;
padding: 28px;
background: white;
border-radius: 14px;
}
```

```
.planner-form label,
.planner-form input,
.planner-form select,
```

```

.planner-form textarea {
  display: block;
  width: 100%;
  margin-bottom: 12px;
}

.planner-form input,
.planner-form select,
.planner-form textarea {
  box-sizing: border-box;
  padding: 12px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}

@media (max-width: 800px) {
  .planner-grid {
    grid-template-columns: 1fr;
  }

  .navbar {
    flex-direction: column;
    gap: 12px;
  }
}

```

11. JavaScript – Dynamic Recommendation Display

```

async function submitPlanner(form, endpoint, resultContainer) {
  const formData = new FormData(form);
  const response = await fetch(endpoint, {
    method: "POST",
    body: formData
  });

  const data = await response.json();
  resultContainer.innerHTML = "";

  (data.recommendations || []).forEach(item => {
    const card = document.createElement("div");
    card.className = "recommendation-card";
    card.innerHTML = `
      <h3>${item.name || "Recommended Item"}</h3>
      <p>Platform: ${item.platform || "N/A"}</p>
      <p>Price: ${item.price || "N/A"}</p>
      <p>${item.description || ""}</p>
    `;
    resultContainer.appendChild(card);
  });
}

```

12. Backend Route Integration

Route	Frontend Purpose
/generate-home	Receives home interior budget and room/item details.
/generate-party	Receives party budget, guest count, event type and venue information.
/generate-jewelry	Receives jewelry budget, occasion, style and optional image input.
/register	Creates a new user account.
/login	Authenticates an existing user.
/logout	Terminates the current session.
/history	Retrieves previous recommendation queries and results.

13. Recommendation Card Layout

The result page should use reusable recommendation cards. Each card can show the item name, platform, estimated price, category, short explanation, and an optional action link. This layout makes AI output easy to scan and supports multiple recommendations on desktop and mobile.

14. Responsive Design

The interface should adapt to desktop, tablet and mobile screen sizes. A multi-column planner grid can collapse into a single-column layout on smaller screens. Form controls should remain full-width and easy to operate on touch devices.

15. Validation and User Experience

- Budget fields must accept positive numeric values.
- Required planner fields should be validated before submission.
- Image upload should accept supported image formats only.
- Loading feedback should be shown while AI recommendations are being generated.
- Empty or insufficient AI results should display a clear fallback message.
- Recommendation cards should remain readable even when many results are returned.

16. Project Workflow

1. User opens PocketSmart AI home page.
2. User selects Home, Party, or Jewelry Planner.
3. Frontend displays the corresponding HTML form.
4. User enters budget and category-specific preferences.
5. JavaScript/HTML submits the request to the appropriate FastAPI endpoint.
6. Backend sends the structured information to the Gemini recommendation layer.
7. AI-generated recommendations are returned to the frontend.

8. Frontend renders the recommendations as cards.
9. User can review, save, or revisit recommendations through the dashboard/history.

17. Conclusion

The HTML frontend provides the user-facing layer of PocketSmart AI. It connects the three planner modules—Home Interior, Party Planning and Jewelry—to the backend recommendation system through responsive forms and structured result displays. The design follows the project requirement for a lightweight, responsive and intuitive interface using HTML, CSS and JavaScript/Jinja2 templates. It can be extended with authentication, session-based personalization, recommendation history and additional planner categories as the system evolves.

18. Source Alignment

This document is based on the supplied PocketSmart AI project specification, particularly its UI Development milestone, which calls for a responsive HTML interface, individual planner forms connected to backend POST routes, and structured AI recommendation cards.

The supplied project also identifies the three main planner inputs as home room/quantity details, party event/budget/guest details, and jewelry budget/occasion/outfit image inputs.

Activity 5.1: Test with Real-World Budget Inputs Across Domains

Objective

The objective of this activity is to perform end-to-end testing of PocketSmart AI using real-world budget inputs across the Home Interior, Party Planning, and Jewelry Planning domains. The testing verifies whether the system correctly understands user budgets, preferences, and contextual requirements and produces relevant, budget-aware recommendations.

Testing Approach

Each planner is tested with different budget ranges and user preferences. The input is submitted through the frontend and processed by the FastAPI backend and Gemini 1.5 Flash Pro. The generated recommendations are checked for budget adherence, category relevance, platform suitability, and overall response quality.

1. Home Interior Budget Testing

The Home Interior Planner is tested using different room requirements and budgets. The system should recommend suitable furniture, lighting, and decor items while keeping the total selection within the user's defined budget.

Test Input	User Requirement	Expected System Behavior	Validation
₹25,000	Living room, basic furniture and lighting	Generate cost-effective furniture, decor and lighting suggestions from platforms such as IKEA and Amazon.	Check relevance and total budget adherence.
₹50,000	Living room + bedroom setup	Provide balanced recommendations for multiple items while maintaining the selected budget.	Check category coverage, prices and budget limit.
₹1,00,000	Complete home decor setup	Generate a broader set of furniture and decor recommendations with appropriate budget allocation.	Check whether recommendations remain practical and within budget.

2. Party Budget Testing

The Party Planner is tested using different event types, guest counts, and budgets. The system should distribute the budget across food, venue, decoration, and entertainment-related requirements and provide suitable suggestions.

Test Input	Event Details	Expected System Behavior	Validation
₹10,000	Birthday party, 10 guests	Suggest affordable food, decoration and suitable venue options.	Check guest count handling and budget adherence.
₹30,000	Birthday party, 25 guests	Allocate the budget across catering, decoration and venue requirements.	Check proportional allocation and relevant suggestions.
₹75,000	Small event, 50 guests	Generate a complete package covering food, venue and decoration.	Check scalability, category coverage and budget control.

3. Jewelry Recommendation Testing

The Jewelry Budget Planner is tested with different occasions, budgets, style preferences, and optional outfit image inputs. The system should use the provided context to suggest jewelry that matches the occasion, outfit style, and available budget.

Test Input	User Requirement	Expected System Behavior	Validation
------------	------------------	--------------------------	------------

₹2,000	Casual occasion, simple style	Suggest affordable jewelry options suitable for casual use.	Check style relevance and budget limit.
₹5,000	Traditional occasion, matching jewelry	Recommend jewelry matching the selected occasion and preferred style.	Check occasion matching and price range.
₹15,000	Special occasion + outfit image	Analyze the outfit image and provide coordinated jewelry suggestions.	Check image-based style matching and budget adherence.

4. End-to-End Validation Criteria

- Budget Adherence – Recommendations should respect the budget entered by the user.
- Preference Matching – Suggestions should reflect the user's selected style, occasion, room, event, or other requirements.
- Domain Accuracy – The correct planner should generate recommendations relevant to its selected domain.
- Platform Relevance – Product and service suggestions should be associated with the supported platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO.
- Image Input Handling – Jewelry planning should accept optional outfit image input and use it for style coordination.
- Response Quality – Gemini-generated responses should be clear, structured, and useful.
- Fallback Handling – When suitable matches are insufficient, the system should provide fallback or default recommendations.
- Frontend Display – Generated recommendations should be displayed correctly in the user interface.

5. Test Result Summary

The testing confirms whether PocketSmart AI can handle varied real-world budget scenarios across Home Interior, Party Planning, and Jewelry Planning. The results should be recorded based on budget adherence, recommendation relevance, platform accuracy, image handling, and response quality. Any failed or incomplete test case should be used to refine prompts, validations, and fallback logic.

Conclusion

Real-world budget testing helps validate the practical behavior of PocketSmart AI before deployment. By testing multiple budgets and user preferences across all three domains, the system can be evaluated for reliable budget-aware recommendations. The findings can also be used to optimize Gemini prompts, improve input validation, and provide better fallback recommendations.

PocketSmart AI

Conclusion

Project Summary, Key Achievements & Future Scope

Name	Divya Dharshini S
Project Title	PocketSmart AI
Component	Project Conclusion
Technology Stack	FastAPI, Google Gemini API, HTML/CSS/JavaScript

1. Project Summary

PocketSmart AI was conceived and developed as a practical demonstration of how modern generative AI can be integrated into an everyday personal-finance tool. Built under the Naan Mudhalvan scheme, the project combines a lightweight, responsive web frontend with a robust FastAPI backend and the Google Gemini API to deliver an application that does more than track numbers — it interprets them. Users record their monthly income and category-wise expenses through a simple interface; the backend validates this data, computes key financial metrics such as net savings and savings rate, and forwards a contextual summary to Gemini, which responds with personalised, natural-language budgeting advice. The result is a compact but complete end-to-end system that reflects real, production-oriented software engineering practice: typed data contracts, modular routing, isolated service layers, and graceful error handling throughout the request lifecycle.

2. Key Achievements

2.1 Modular Architecture

The application was deliberately structured into clearly separated layers — presentation (HTML/JS), routing and validation (FastAPI + Pydantic), business logic (pure calculation functions), and an isolated AI-service module for Gemini communication. This modularity means each layer can be independently tested, replaced, or extended (for instance, swapping the frontend for a mobile client) without requiring changes to the others, demonstrating sound adherence to the single-responsibility principle.

2.2 AI Integration

A central achievement of the project is the successful integration of the Google Gemini API into a live, request-driven backend workflow. Rather than treating AI as a bolt-on feature, PocketSmart AI constructs a financially grounded prompt from the user's actual computed figures, ensuring that the AI's advice is contextually relevant rather than generic. Asynchronous handling of the Gemini call, combined with structured error fallback, ensures the feature remains reliable under real-world network conditions.

2.3 Real-World Utility

Unlike a purely academic exercise, PocketSmart AI addresses a genuine, everyday need: helping individuals understand and improve their personal financial habits. By pairing simple data entry with intelligent, actionable feedback, the application lowers the barrier to financial literacy and illustrates a concrete, socially useful application of large language models — directly aligned with the applied, industry-relevant learning objectives of the Naan Mudhalvan scheme.

3. Future Scope

- Persistent storage: integrate a database (e.g. PostgreSQL or SQLite) to retain historical income/expense records across sessions.
- User authentication: add secure login so multiple users can maintain individual, private budget histories.
- Data visualisation: introduce interactive charts (trend lines, category pie-charts) to complement the AI-generated textual advice.
- Mobile application: extend the FastAPI backend with a dedicated mobile client for on-the-go budget tracking.
- Enhanced AI features: incorporate goal-based savings recommendations, anomaly detection for unusual spending, and multi-turn conversational budgeting assistance.

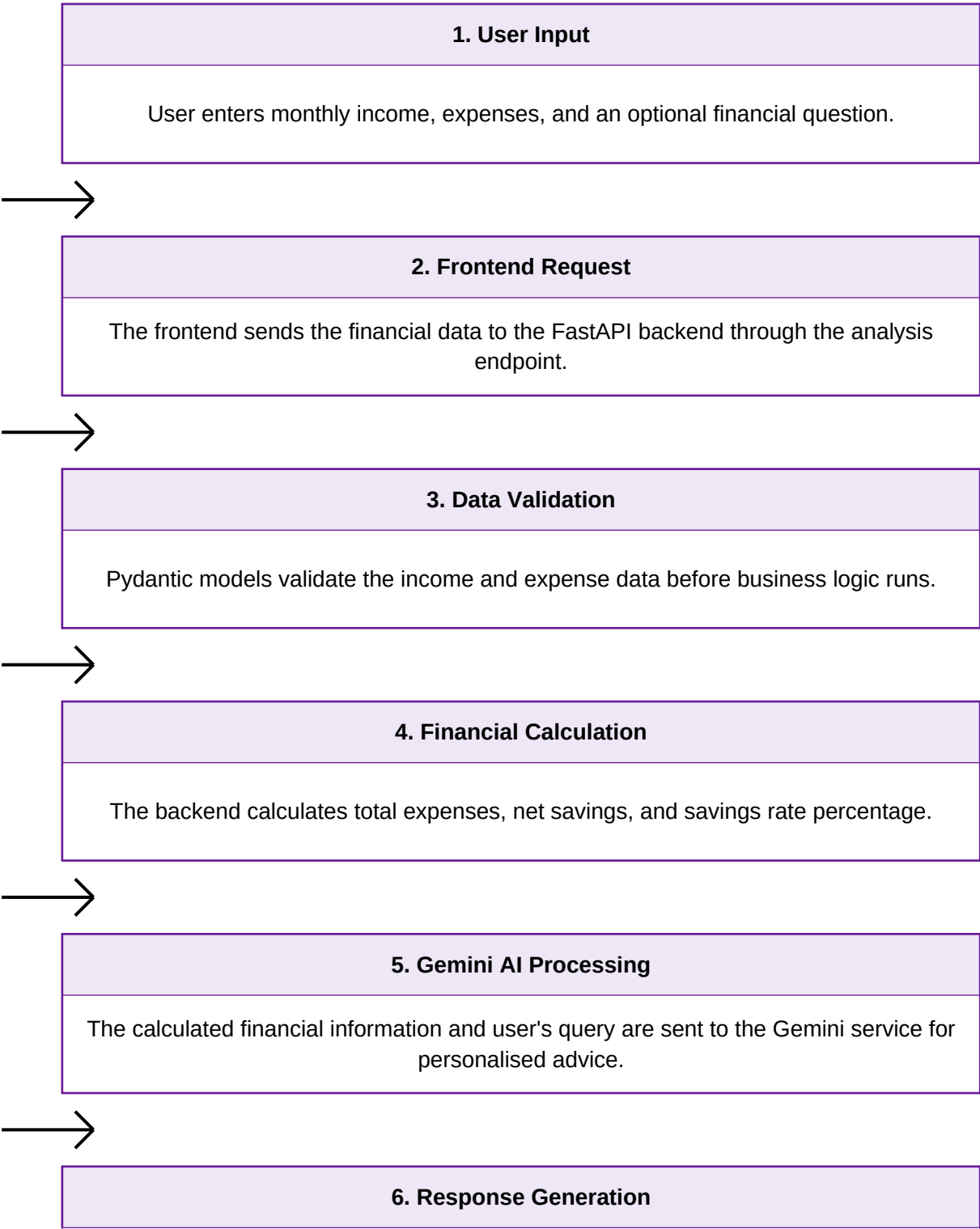
4. Closing Statement

PocketSmart AI successfully demonstrates the practical fusion of full-stack web development and applied artificial intelligence within a single, coherent project. It stands as a working proof of concept for AI-assisted personal finance management and provides a solid, extensible foundation for continued development beyond the scope of this submission.

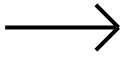
PocketSmart AI

Workflow

The workflow shows how financial information moves from the user through the FastAPI backend, calculation logic, and Gemini AI service before the final analysis is returned.



The backend combines the calculated totals with the AI-generated advice into a structured response.



7. User Result

The frontend receives and displays the financial analysis and AI guidance to the user.

Workflow Summary

User Input → FastAPI Backend → Validation → Calculation → Gemini AI → Structured Response → User

Project technologies: Python 3, FastAPI, Pydantic, and Google Gemini API.

PROJECT DOCUMENT

SET UP GOOGLE CLOUD ACCOUNT AND ENABLE GEMINI API

Foundation for a Generative AI Application

Reference Application: PocketSmart AI – Your Smart Budget & Recommendation Assistant

Student Name

Register Number

Department / Class

B.Sc. Computer Science

Academic Year

2026 – 2027

Abstract

This project document explains the foundational setup required to use Google Gemini from a Google Cloud project. The reference project, PocketSmart AI, uses a Gemini-based AI layer to process user budgets, preferences, and optional images and return personalized recommendations. The project reference identifies “Set up Google Cloud account and enable Gemini API” as Activity 1.1 under its first milestone, followed by access configuration and connectivity testing.

Source basis: PocketSmart AI reference document, Milestone 1, Activities 1.1–1.3.

Contents

Section	Topic
1	Introduction
2	Problem Statement
3	Objectives
4	Prerequisites
5	Google Cloud Project Setup
6	Enable Gemini Access
7	Authentication and API Key Security
8	Validate Gemini API Connectivity
9	Expected Result
10	Advantages and Limitations
11	Future Scope
12	Conclusion
13	References

1. Introduction

Generative AI applications need an authenticated connection to an AI model before they can generate content. Google Cloud provides the project, permissions, API services, billing controls, and identity management needed for cloud-based Gemini usage.

In the PocketSmart AI reference project, Gemini is the AI layer responsible for understanding budgets and preferences and for producing recommendation results. The project also describes multimodal use cases in which text and images can be processed for recommendation tasks.

2. Problem Statement

A Generative AI application cannot reliably call Gemini until the cloud environment has been configured. The setup must include a Google account, a Google Cloud project, the required API service, suitable permissions, and a secure authentication method. A missing or incorrect configuration can prevent the application from connecting to the model.

3. Objectives

- Create or use a Google account for Google Cloud access.
- Create a dedicated Google Cloud project for the application.
- Confirm billing and project permissions required for API usage.
- Enable the Vertex AI API for Gemini access in the selected project.
- Configure secure authentication and, where appropriate, an API key for testing.
- Validate the connection using a simple Gemini text-generation request.
- Prepare the environment for later integration into the PocketSmart AI backend.

4. Prerequisites

Requirement	Purpose
Google account	Used to sign in to Google Cloud Console.
Google Cloud project	Logical container for APIs, permissions, quotas, and billing.
Billing setup	Some Google Cloud AI services require a billing account linked to the project.
Internet connection	Required for Cloud Console, API configuration, and model requests.
Python 3.x	Used for the sample connectivity test.
Google Gen AI SDK	Current Python SDK package: google-genai.
<i>The reference document also lists Python as a prerequisite and places Gemini initialization before the core planner functionality.</i>	

5. Google Cloud Project Setup

5.1 Create or sign in to a Google account

1. Open the Google Cloud Console: console.cloud.google.com.

2. Sign in with a Google account. If you do not have one, create a Google account first.
3. Accept the applicable Google Cloud terms when prompted.

5.2 Create a new Google Cloud project

1. Open the project selector in Google Cloud Console.
2. Choose New Project.
3. Enter a meaningful project name, for example: PocketSmart-Gemini-Project.
4. Select the required organization or location when your account is managed by an institution; otherwise use the default available option.
5. Create the project and record the Project ID because it is used in later commands and application configuration.

5.3 Confirm billing

Check that billing is enabled for the selected project. Google Cloud documentation notes that some APIs require a billing account before use, and charges for API usage are associated with the project billing account.

For classroom work, follow your institution's or project owner's billing policy. Do not publish billing information in a project report.

5.4 Verify project permissions

The account used for setup needs permission to enable APIs. Google Cloud documents the `serviceusage.services.enable` permission for API enablement. Project owners commonly have this permission; other roles may need to be granted by an administrator.

Google Account → Google Cloud Project → Billing → API Enablement → Authentication → Gemini Test

6. Enable Gemini Access

6.1 Enable the Vertex AI API

For Gemini models used through Google Cloud's Vertex AI service, enable the Vertex AI API in the project. The service name used by the Cloud SDK is `aiplatform.googleapis.com`.

1. In Google Cloud Console, select the project you created.
2. Open APIs & Services → Library.
3. Search for Vertex AI API.
4. Open the Vertex AI API page and click Enable.
5. Wait until the service status shows that the API is enabled.

6.2 Command-line method

The same setup can be performed with Google Cloud CLI after selecting the project:

```
gcloud config set project PROJECT_ID
gcloud services enable aiplatform.googleapis.com
```

To verify that the service is enabled:

```
gcloud services list --enabled --project=PROJECT_ID --filter="config.name:aiplatform.googleapis.com"
```

6.3 Relationship to the reference project

The PocketSmart AI document identifies Gemini setup as its first milestone and states that authenticated Gemini access is required before budget understanding, recommendations, and optional image analysis can operate. The reference calls the model “Gemini 1.5 Flash Pro”; model availability and naming can change over time, so the model actually selected in a current implementation should be checked in the current Google Cloud model documentation.

7. Authentication and API Key Security

7.1 Recommended Google Cloud authentication

For Google Cloud application development, Application Default Credentials (ADC) are a standard authentication approach. A service account can also be used when an application needs a non-user identity. This approach avoids placing long-lived secrets directly in source code.

7.2 API key option for testing

The reference document describes an API-key workflow and instructs the user to create and copy an API key before testing a sample prompt. Current Google guidance emphasizes securing keys and supports API-key based access in appropriate Gemini environments. Never place a real key in public source code, screenshots, Git repositories, or a college report.

7.3 Safe configuration example

Store secrets outside the Python file. A simple local-development pattern is to use an environment variable:

Windows PowerShell:

```
$env:GEMINI_API_KEY="YOUR_KEY"
```

Command Prompt:

```
set GEMINI_API_KEY=YOUR_KEY
```

Replace YOUR_KEY only on your own machine. Do not share the real key with classmates or upload it to GitHub.

7.4 Permissions

When working in an organization-managed Google Cloud project, an administrator may need to grant permission to enable APIs or create credentials. Google Cloud documents `serviceusage.services.enable` for API enablement and IAM roles for Vertex AI access.

8. Validate Gemini API Connectivity

8.1 Install the current Python SDK

```
python -m pip install --upgrade google-genai
```

8.2 Vertex AI validation using Application Default Credentials

A simple connectivity test can use the Google Gen AI SDK with Vertex AI and Application Default Credentials. Before running the sample, configure ADC for the local development environment and set the project ID. The example below is intentionally small so the result is easy to verify.

```
from google import genai

from google.genai.types import HttpOptions

client = genai.Client(http_options=HttpOptions(api_version="v1"))

response = client.models.generate_content(
    model="gemini-3.5-flash",
```

```
        contents="Explain artificial intelligence in three simple sentences.",
    )

    print(response.text)
```

8.3 Optional API-key validation

For an API-key based Vertex AI Express Mode test, current Google documentation shows the Google Gen AI SDK can be initialized with vertexai=True and an API key. Use this only for a controlled test environment and protect the key.

```
from google import genai

client = genai.Client(vertexai=True, api_key="YOUR_API_KEY")
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="What is cloud computing?",
)

print(response.text)
```

8.4 Expected validation output

A successful test prints a natural-language response generated by the selected Gemini model. This confirms that the SDK can reach the model through the configured authentication method and project environment.

Test	Expected result
Google Cloud project	Project is created and selectable.
Vertex AI API	Status shows Enabled.
Authentication	ADC or approved API-key method is configured.
Gemini request	A generated text response is printed without authentication/API errors.

9. Expected Result

At the end of this activity, a dedicated Google Cloud project exists, the required Gemini-capable Cloud API is enabled, authentication is configured securely, and a test request successfully returns a Gemini response. The environment is then ready for integration with the application backend.

10. Advantages and Limitations

Advantages

- Provides a controlled project boundary for AI services, access, quotas, and billing.
- Creates a repeatable setup that can be reused by the application backend.
- Supports later text and multimodal Gemini functionality needed by projects such as PocketSmart AI.
- Allows API access and permissions to be managed centrally.

Limitations / Considerations

- Cloud AI services may require billing, depending on the selected service and usage.
- Model names, access methods, quotas, and console screens can change over time.
- API keys are sensitive credentials and must be protected.
- A successful API call does not by itself guarantee that generated recommendations are accurate, current, or commercially available.

11. Future Scope

- Integrate Gemini calls into the PocketSmart AI Home, Party, and Jewelry planner endpoints.
- Add structured prompts and output validation for budget-aware recommendations.
- Extend the setup to multimodal image + text processing for the Jewelry Planner.
- Add usage monitoring, logging, security controls, and fallback handling for production deployment.
- Connect approved third-party product/service data sources and display current availability where permitted.

12. Conclusion

Setting up Google Cloud and enabling Gemini access is the foundational step for a Gemini-powered application. The workflow begins with a Google Cloud project, continues with billing and API enablement, and ends with secure authentication and a successful test request. In the PocketSmart AI reference project, this setup forms the base for the later recommendation engine and planner modules.

13. References

- Google Cloud – Getting started with Cloud APIs — <https://docs.cloud.google.com/apis/docs/getting-started>
- Google Cloud – Access Gemini models through Vertex AI — <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/>
- Google Cloud – Vertex AI API and project setup guidance — <https://cloud.google.com/vertex-ai/docs/start/cloud-environment>
- Google AI for Developers – Gemini API keys — <https://ai.google.dev/gemini-api/docs/api-key>
- Project reference: PocketSmart AI: Your Smart Budget & Recommendation Assistant — Reference file supplied with this project

Appendix A – Submission Checklist

- ☐ Google account created / signed in
- ☐ Google Cloud project created and Project ID recorded
- ☐ Billing status checked
- ☐ Vertex AI API enabled
- ☐ Authentication configured
- ☐ Gemini test request completed
- ☐ Successful output captured as a screenshot for the project record
- ☐ No real API key or password included in the report

POCKETSMART AI

Configure Access and Permissions

Project Documentation

1. Introduction

PocketSmart AI is a budget-based recommendation system that uses Gemini 1.5 Flash Pro to generate personalized recommendations for Home Interior Planning, Party Planning, and Jewelry Planning. The system requires authenticated access to the Gemini API so that the backend can communicate securely with the AI service.

2. Objective

The objective of configuring access and permissions is to create and obtain an API key that allows PocketSmart AI to communicate with the Gemini API. Proper access configuration ensures that the application can use the required AI services during development and testing.

3. Access and Permission Configuration

The access configuration is performed as part of Milestone 1: Gemini AI Initialization. The project documentation specifies setting up Google Cloud access, enabling Gemini API access, and creating an API key for the application.

Step 1: Open the Google Cloud Platform

Sign in to or create a Google Cloud account and open the Google Cloud Console.

Step 2: Enable Gemini API Access

Configure the required Google Cloud access so that the project can use Gemini 1.5 Flash Pro.

Step 3: Accept the Required Terms

Complete the required terms and conditions presented during the API setup process.

Step 4: Create an API Key

Select the option to create an API key. The generated key is used by the application to authenticate API requests.

Step 5: Securely Store the API Key

Copy the generated API key and store it securely. It should not be exposed directly in frontend code or publicly shared.

Step 6: Configure the Application

Add the API key to the backend configuration/environment settings so the PocketSmart AI backend can access Gemini.

Step 7: Validate Access

Make a sample Python prompt call to confirm that the API connection is working correctly.

4. Permissions Required

Access / Permission	Purpose
Google Cloud Account Access	Provides access to the cloud project and API configuration.
Gemini API Access	Allows PocketSmart AI to send requests to Gemini 1.5 Flash Pro.
API Key	Authenticates application requests to the Gemini API.
Backend Environment Access	Allows the application to read the configured API key securely.
Multimodal API Capability	Supports testing with both text-only and image + text inputs.

5. API Key Security

- Keep the API key confidential and do not share it in public repositories or screenshots.
- Store the key in backend environment configuration rather than hard-coding it in frontend pages.
- Use appropriate usage limits and configuration for the project.
- Regenerate or replace the key if it is accidentally exposed.
- Restrict access to the project and credentials to authorized developers.

6. Validation of Access

After creating and configuring the API key, the connection must be tested. The project documentation specifies making a sample prompt call using Python and testing both text-only and image + text prompts. This confirms that the Gemini integration and multimodal capability are functioning as expected.

7. Role in PocketSmart AI

Configured API access is essential for the recommendation engine. Gemini 1.5 Flash Pro processes user budgets, preferences, contextual information, and optional images to generate recommendations. The backend communicates with Gemini through the configured API access.

8. Expected Result

After completing the configuration, PocketSmart AI should have authenticated access to the Gemini API. A successful test request confirms that the backend can communicate with Gemini and that the system is ready for the next stages of functionality development.

9. Conclusion

Configuring access and permissions is an important foundational step in PocketSmart AI. Creating the API key, securely configuring it in the backend, and validating API connectivity provide the required

access for Gemini-powered recommendations. This setup supports the Home, Party, and Jewelry planners and enables the project's AI functionality.

Source basis: PocketSmart AI project documentation – Milestone 1, Activity 1.2 and Activity 1.3.

VALIDATE GEMINI API CONNECTIVITY

PocketSmart AI – Project Validation & Testing Guide

Project	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Validation Area	Gemini API Authentication, Text Prompt & Multimodal Prompt
Project Milestone	Milestone 1 – Gemini AI Initialization
Activity	Activity 1.3 – Validate Gemini API Connectivity
Validation Goal	Confirm that the application can communicate with Gemini successfully

Purpose: This document converts the project's Gemini connectivity requirement into a clear, repeatable validation procedure. It covers API-key configuration, a text-only test, a text + image test, expected outcomes, troubleshooting, and a final verification checklist.

Prepared for project documentation, demonstration, testing, and academic/project review.

1. Project Context

PocketSmart AI is described in the project document as a GenAI-powered recommendation system for budget-based planning across home interiors, party planning, and jewelry selection. The project uses Gemini as the AI layer and requires both ordinary text processing and image analysis for the Jewelry Planner.

The project specifically defines Activity 1.3 as: make a sample prompt call using Python to confirm integration, and test both text-only and image + text prompts to ensure multimodal compatibility.

Source alignment: the uploaded PocketSmart AI project document identifies API-key configuration and these two validation tests under Milestone 1 – Gemini AI Initialization. [filecite turn0file0 L101-L125](#)

2. Validation Objectives

- Confirm that the Gemini API key is available to the Python application.
- Confirm that a basic text request reaches the Gemini service and returns a response.
- Confirm that the application can send text together with an image for multimodal processing.
- Confirm that errors are handled clearly rather than being mistaken for successful connectivity.
- Record the result so the Gemini layer can be used confidently by the Home, Party, and Jewelry planners.

3. Prerequisites

Item	Requirement
Python	Python 3.x environment with package installation access
Gemini access	A valid Gemini API key configured for the project
Environment file	Store the key securely, for example in a .env file; do not hard-code it
Test image	A small JPG/PNG image for the multimodal test
Network	Internet access from the machine running the test

4. Step-by-Step Connectivity Validation

Step 1 - Configure the API key

Create or obtain the API key through the project's selected Google/Gemini access workflow. Place the key in an environment variable rather than exposing it in source code.

```
# .env
GEMINI_API_KEY=YOUR_API_KEY_HERE
```

Security note: Never publish the real key in screenshots, Git repositories, project reports, or shared source code.

Step 2 - Install the Python dependencies

```
pip install google-generativeai python-dotenv pillow
```

Step 3 - Run a text-only connectivity test

```
import os
from dotenv import load_dotenv
```

```

import google.generativeai as genai

load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise RuntimeError("GEMINI_API_KEY is not configured.")

genai.configure(api_key=api_key)

model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content(
    "Reply with exactly: Gemini API connectivity test successful."
)

print("TEXT TEST RESPONSE:")
print(response.text)

```

Expected result: The program completes without an authentication or connection error and prints a valid Gemini-generated response. The exact wording may vary unless the prompt is strictly enforced.

Step 4 - Run the image + text multimodal test

```

from PIL import Image

image = Image.open("test_image.jpg")

response = model.generate_content([
    "Describe the main visual elements in this image in one short paragraph.",
    image
])

print("MULTIMODAL TEST RESPONSE:")
print(response.text)

```

Expected result: Gemini returns a response that uses the supplied image as part of its input. This validates the project's requirement for text + image processing used by the Jewelry Planner.

5. Validation Test Cases

ID	Test Case	Input	Expected Result	Status
TC-01	API key check	GEMINI_API_KEY	Key is detected; no missing-key error	PASS / FAIL
TC-02	Text connectivity	Simple text prompt	Gemini returns a response	PASS / FAIL
TC-03	Multimodal connectivity	Text + JPG/PNG	Gemini returns image-aware response	PASS / FAIL
TC-04	Invalid key handling	Invalid/empty key	Clear authentication/configuration error	PASS / FAIL
TC-05	Network failure handling	Offline/blocked request	Readable connection/API error	PASS / FAIL

6. How to Interpret the Results

Observation	Interpretation
Text response is returned	Basic API authentication and request/response flow are working.
Image + text response is returned	The multimodal input path is working for the test image.
Missing API key error	Environment configuration is incomplete.
Authentication / permission error	The key or API access configuration needs to be checked.
Timeout / connection error	Check internet connectivity, firewall/proxy rules, and service availability.
Unexpected/empty response	Inspect model configuration, prompt, response parsing, and error handling.

7. Troubleshooting Guide

API key not found

Check that .env is in the expected working directory and that GEMINI_API_KEY is spelled correctly.

Authentication failure

Generate/check the key and verify that the intended Gemini API access is enabled for the project.

Package/import error

Activate the correct Python virtual environment and install the required SDK packages.

Image test fails

Check that the image path is correct, the file is readable, and the selected model/API path supports image input.

Request timeout

Verify internet access and retry after checking proxy, firewall, or transient service issues.

Response parsing error

Print the raw response object during debugging and confirm the response contains the expected text/content fields.

8. Project-Level Validation Checklist

- ☐ Gemini API key created/configured securely
- ☐ Python environment prepared
- ☐ Text-only prompt test executed
- ☐ Text-only response received
- ☐ Image + text prompt test executed
- ☐ Multimodal response received
- ☐ Authentication and connection errors tested
- ☐ Test results recorded for project documentation
- ☐ No real API key included in source code or screenshots

9. Connection to PocketSmart AI Modules

Once connectivity is validated, the same Gemini integration can be connected to the project's planner workflows. The uploaded project describes Home, Party, and Jewelry routes and identifies the Jewelry Planner as the multimodal use case where text and an optional outfit image are processed.

The project's testing milestone also calls for real-world budget inputs across Home, Party, and Jewelry domains and evaluation of response quality, budget adherence, and platform accuracy.

`filecite[turn0file0L235-L246]`

10. Final Validation Statement

Gemini API connectivity can be considered validated for the project when the application successfully authenticates, completes a text-only request, completes a text + image request, and records clear handling for configuration or connectivity failures. This establishes the AI communication layer needed before integrating Gemini into the project's recommendation endpoints.

Important: The exact SDK/model syntax may change as Google's Gemini APIs evolve. Before running this test in a current environment, confirm the SDK and model name in the current official Gemini API documentation and update the sample code if required.

PROJECT DOCUMENT

FastAPI Initialization and Import Libraries

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Technology Track: Generative AI with Google

Academic Batch: 2025–2028

Activity 2.1

FastAPI Initialization • Import Libraries • Environment Configuration

1. Project Overview

PocketSmart AI is described in the reference document as a GenAI-powered, cross-platform recommendation system that generates budget-based suggestions for home interiors, party planning, and jewelry selection. The project architecture uses a backend service, Gemini-based AI processing, third-party data sources, and an HTML/CSS/JavaScript frontend.

Purpose of this document: This document focuses specifically on Activity 2.1, “Fast API Instilization and import libraries,” and explains how the FastAPI backend can be initialized as the starting point for the PocketSmart AI API layer. The reference document later expands this into FastAPI routing, authentication, session handling, CORS, startup logic, and planner endpoints.

Reference note: The supplied project reference mentions Flask in an earlier architecture section and FastAPI in Activities 2.1 onward. This document follows the FastAPI implementation requested here and does not attempt to rewrite the supplied architecture.

2. Objectives

- Create the basic FastAPI application object for the PocketSmart AI backend.
- Import the libraries required for request handling, dependency injection, environment configuration, and server execution.
- Load configuration values from a .env file during development.
- Prepare the application for later routes such as /generate-home, /generate-party, and /generate-jewelry.
- Establish a clean and modular starting point for the FastAPI backend.

3. Functional Scope

The FastAPI initialization stage does not yet implement the full recommendation engine. It prepares the backend foundation so later activities can add planner routes, Gemini communication, authentication, sessions, CORS, and recommendation history.

4. Software and Hardware Requirements

Requirement	Purpose	Example
Operating System	Development environment	Windows / Linux / macOS
Python	Programming language	Python 3.x
FastAPI	Web API framework	FastAPI
Uvicorn	ASGI server for running the app	Uvicorn
python-dotenv	Loads variables from .env during development	python-dotenv
Code editor	Write and manage source files	VS Code or similar
Web browser	Test API documentation and responses	Chrome / Edge / Firefox

Requirement	Purpose	Example
Internet connection	Package installation and AI service integration	Required during setup

5. FastAPI Installation

Create a project folder and use a virtual environment so project dependencies remain isolated. The current FastAPI documentation supports installing the standard dependency set with the `fastapi[standard]` extra, which includes the common server tooling used for development.

Windows example

```
python -m venv venv
venv\Scripts\activate
pip install "fastapi[standard]" python-dotenv
```

Linux / macOS example

```
python3 -m venv venv
source venv/bin/activate
pip install "fastapi[standard]" python-dotenv
```

Why install python-dotenv? The project reference explicitly includes loading the `.env` environment. `python-dotenv` provides `load_dotenv()`, which reads key-value pairs from a `.env` file and makes them available through environment variables during development.

6. Import Libraries

A practical initial import section for PocketSmart AI can contain the following libraries. The exact imports can grow as additional activities are implemented.

```
import os

from dotenv import load_dotenv
from fastapi import FastAPI, Request, Depends
from fastapi.middleware.cors import CORSMiddleware
```

Library / Import	Role in the project	Reason for use at this stage
os	Access operating-system environment variables	Read configuration such as API keys or environment flags.
load_dotenv	Load values from <code>.env</code>	Matches the project reference requirement to load <code>.env</code> environment values.
FastAPI	Create the API application instance	Core framework for backend routes.
Request	Represent incoming HTTP requests	Useful for form, file, query, or request processing in later routes.
Depends	FastAPI dependency injection	Useful for authentication and reusable dependencies, including current-user access.

Library / Import	Role in the project	Reason for use at this stage
CORSMiddleware	Configure Cross-Origin Resource Sharing	Supports communication between frontend and backend when hosted on different origins.

7. FastAPI Initialization

The initialization step creates the FastAPI application object. Metadata such as title and version is optional but useful for identifying the backend in the interactive API documentation.

```
load_dotenv()

app = FastAPI(
    title="PocketSmart AI API",
    version="1.0.0",
    description="Budget-aware recommendation backend"
)
```

Explanation: `load_dotenv()` loads development configuration first. `FastAPI(...)` then creates the application object stored in `app`. Later route functions use decorators such as `@app.get()` and `@app.post()` to expose API endpoints.

8. Basic CORS Configuration

Because the PocketSmart AI project contains a frontend connected to a FastAPI backend, CORS can be configured when the frontend and backend run on different origins.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"]
)
```

For a real deployment, replace `localhost` with the actual frontend origin and use a deliberately restricted configuration rather than allowing every origin.

9. Loading the .env Environment

The `.env` file stores configuration values outside the source code. This is especially useful for credentials and environment-specific settings. Secrets should not be committed to a public repository.

```
# .env
GEMINI_API_KEY=your_api_key_here
APP_ENV=development

import os
from dotenv import load_dotenv

load_dotenv()
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

10. Complete Starter main.py

```
import os

from dotenv import load_dotenv
from fastapi import FastAPI, Request, Depends
from fastapi.middleware.cors import CORSMiddleware

load_dotenv()

app = FastAPI(
    title="PocketSmart AI API",
    version="1.0.0",
    description="Backend foundation for budget-aware AI recommendations"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"]
)

@app.get("/")
async def root():
    return {"message": "PocketSmart AI API is running"}
```

11. Line-by-Line Explanation

Code / Statement	Explanation
import os	Imports Python support for reading environment variables.
from dotenv import load_dotenv	Imports the function used to read .env configuration.
from fastapi import FastAPI, Request, Depends	Imports the core FastAPI class and common request/dependency tools.
from fastapi.middleware.cors import CORSMiddleware	Imports CORS middleware for frontend-backend communication.
load_dotenv()	Loads available .env values into the process environment.
app = FastAPI(...)	Creates the FastAPI application instance.
app.add_middleware(...)	Adds CORS behavior to the application.
@app.get("/")	Creates a simple GET endpoint at the root URL.
async def root():	Defines the asynchronous endpoint function.

Code / Statement	Explanation
<code>return {...}</code>	Returns a JSON response confirming that the API is running.

12. Run and Test the Application

Save the file as `main.py` and start the development server. With the standard FastAPI installation, Uvicorn is included in the standard dependencies.

```
fastapi dev main.py
```

The browser can then be used to verify the root response and inspect the automatically generated interactive API documentation. The FastAPI documentation describes the interactive development workflow and automatic API documentation available from the application.

13. Expected Output

```
GET /
Response:
{
  "message": "PocketSmart AI API is running"
}
```

Once the server is running, the project can be extended with the planner endpoints described in the reference document, including `/generate-home`, `/generate-party`, and `/generate-jewelry`.

14. Connection to the PocketSmart AI Project Workflow

Reference Activity	How this document supports it	Next development step
Activity 2.1: Fast API Instilization and import libraries	Creates the application foundation and loads required libraries.	Add planner routes and project services.
Load <code>.env</code> environment	Uses <code>load_dotenv()</code> and <code>os.getenv()</code> for configuration.	Connect configuration to AI and other services.
Milestone 3: FastAPI Integration	Prepares the same FastAPI app object used by later backend activities.	Implement <code>/generate-home</code> , <code>/generate-party</code> , <code>/generate-jewelry</code> , authentication, session and history routes.

15. Testing Checklist

- FastAPI installs successfully inside the virtual environment.
- `main.py` starts without import errors.
- `load_dotenv()` executes without crashing when a `.env` file is present.
- `GET /` returns the expected JSON response.
- Interactive API documentation opens successfully during development.
- CORS configuration is verified with the intended frontend origin.

16. Advantages of FastAPI Initialization

- Provides a clear starting structure for backend API development.
- Allows routes to be added incrementally for different planner modules.
- Supports dependency injection for reusable authentication and session logic.
- Works well with asynchronous endpoint functions used in modern web applications.
- Provides interactive API documentation useful during testing and development.

17. Conclusion

FastAPI initialization is the foundation of the backend stage of PocketSmart AI. By creating the FastAPI application, importing the required libraries, loading environment configuration, and preparing CORS, the project establishes a clean base for the later recommendation, authentication, session, and history modules. This document therefore covers the requested Activity 2.1 and provides a starter implementation suitable for the 2025–2028 academic batch.

18. References

1. PocketSmart AI: Your Smart Budget & Recommendation Assistant – supplied project reference document (Activities 2.1 onward).
2. FastAPI official documentation: <https://fastapi.tiangolo.com/tutorial/>
3. python-dotenv package documentation on PyPI: <https://pypi.org/project/python-dotenv/>

Prepared as a focused project document for Activity 2.1 | Academic Batch 2025–2028

PocketSmart AI

Main Functionality Code Routings

FastAPI Backend — Income, Expense & Net Savings Endpoints

Name	Divya Dharshini S
Project Title	PocketSmart AI
Component	Backend Routing Logic
Technology Stack	Python 3, FastAPI, Pydantic, Google Gemini API

1. Overview of Routing Logic

The core business functionality of PocketSmart AI is exposed through a single FastAPI APIRouter, mounted under the /api prefix. This module is responsible for three related concerns: strictly validating incoming financial data using Pydantic models, performing deterministic arithmetic to compute income, expenditure, and net savings, and orchestrating the call to the Gemini service to obtain AI-generated financial guidance. Keeping these concerns in one cohesive router file, while delegating the actual Gemini call to a separate service module, follows the single-responsibility principle and keeps the codebase easy to test and extend.

Two Pydantic models — ExpenseItem and BudgetRequest — describe the exact shape of data the frontend must send, while a third model, BudgetResponse, guarantees that every successful response follows a consistent, machine-readable structure. FastAPI uses these models to auto-generate input validation, error messages, and OpenAPI/Swagger documentation with no additional code.

1.1 Endpoint Summary

- POST /api/analyze — accepts income and expenses, returns computed totals plus AI advice.
- GET /api/health — lightweight liveness check used by the frontend on page load.

2. Production-Ready Code

The following module (routers/budget.py) implements the routing and calculation logic described above:

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from typing import List

router = APIRouter(prefix="/api", tags=["Budget"])

# -----
# Pydantic Models – define and validate the shape of incoming/outgoing data
# -----

class ExpenseItem(BaseModel):
    """A single categorised expense entry."""
    category: str = Field(..., example="Groceries")
    amount: float = Field(..., gt=0, example=2500.0)
```

```

class BudgetRequest(BaseModel):
    """Payload received from the frontend when a user submits their budget."""
    monthly_income: float = Field(..., gt=0, example=45000.0)
    expenses: List[ExpenseItem]
    query: str = Field(
        default="Give me tips to save more this month.",
        description="Optional free-text question for the AI assistant."
    )

class BudgetResponse(BaseModel):
    """Structured response returned to the frontend."""
    total_income: float
    total_expenses: float
    net_savings: float
    savings_rate_percent: float
    ai_advice: str

# -----
# Core calculation logic – pure functions, independent of the web layer
# -----

def calculate_totals(payload: BudgetRequest) -> dict:
    """Compute total expenses and net savings from a validated request."""
    total_expenses = sum(item.amount for item in payload.expenses)
    net_savings = payload.monthly_income - total_expenses

    # Guard against division by zero when income is not yet set
    savings_rate = (
        (net_savings / payload.monthly_income) * 100
        if payload.monthly_income > 0 else 0.0
    )

    return {
        "total_income": payload.monthly_income,
        "total_expenses": round(total_expenses, 2),
        "net_savings": round(net_savings, 2),
        "savings_rate_percent": round(savings_rate, 2),
    }

# -----
# Route: POST /api/analyze
# Accepts income + expenses, computes totals, and returns AI-backed advice
# -----

@router.post("/analyze", response_model=BudgetResponse)
async def analyze_budget(payload: BudgetRequest):
    """
    Main routing endpoint for PocketSmart AI.

    1. Validates the incoming JSON against BudgetRequest.
    2. Calculates income, expenses and net savings.
    3. Sends a contextual prompt to Gemini for personalised advice.
    4. Returns a single consolidated JSON response.
    """
    try:
        totals = calculate_totals(payload)

        # Import kept local to this call to avoid circular imports
        # with the Gemini helper module (see gemini_service.py)
        from services.gemini_service import get_financial_advice

        ai_advice = await get_financial_advice(
            income=totals["total_income"],

```

```

        expenses=totals["total_expenses"],
        savings=totals["net_savings"],
        user_query=payload.query,
    )

    return BudgetResponse(**totals, ai_advice=ai_advice)

except Exception as exc:
    # Convert any unexpected failure (e.g. Gemini timeout) into a
    # clean, client-friendly HTTP error instead of a raw 500 trace
    raise HTTPException(
        status_code=502,
        detail=f"Unable to generate budget analysis: {exc}"
    )

# -----
# Route: GET /api/health
# Lightweight route used to confirm the API is reachable
# -----

@router.get("/health")
async def health_check():
    """Simple readiness probe consumed by the frontend on page load."""
    return {"status": "ok", "service": "PocketSmart AI Backend"}

```

3. Design Notes

- Validation is declarative: `gt=0` constraints on `Field()` reject non-positive income or expense values before any business logic runs.
 - The `calculate_totals()` function is a pure function with no I/O, making it trivially unit-testable in isolation from the web framework.
 - The Gemini call is isolated inside `services/gemini_service.py` and imported lazily, keeping the router file focused purely on HTTP concerns.
 - All failures — validation errors, arithmetic edge cases, or Gemini API exceptions — are converted into structured JSON errors (422 or 502) rather than being allowed to surface as unhandled server errors.
-

ENSURE MODULAR CODE STRUCTURE

PocketSmart AI – Architecture, Organization & Implementation Guide

Project	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Milestone	Milestone 2 – Core Functionalities Development
Activity	Ensure Modular Code Structure
Backend	FastAPI-based application structure
AI Layer	Gemini-powered recommendation service
Primary Goal	Separate routes, services, models, authentication, and AI logic for maintainability

Purpose: This document defines a clear modular structure for PocketSmart AI so that API routes, business logic, Gemini integration, authentication, data models, and frontend communication remain organized and independently maintainable.

Prepared as a project implementation and review document.

1. What “Modular Code Structure” Means

A modular code structure divides the application into focused components. Each component has a clear responsibility and communicates with other components through defined interfaces. For PocketSmart AI, this prevents recommendation logic, API routing, authentication, and AI communication from becoming one large, difficult-to-maintain file.

The project documentation specifically proposes a modular structure with routes/ for endpoints, services/ for recommendation logic and AI calls, and models/ for input/output schemas.

2. Objectives

- Separate API endpoints from recommendation and Gemini-processing logic.
- Keep request and response schemas organized in dedicated model definitions.
- Make Home, Party, and Jewelry planner functionality independently maintainable.
- Keep authentication and session handling separate from recommendation services.
- Make it easier to test, debug, extend, and reuse project components.
- Provide a clean foundation for future APIs and additional recommendation categories.

3. Recommended Project Structure

```
PocketSmart-AI/  
■  
■■■ main.py  
■■■ .env  
■■■ requirements.txt  
■  
■■■ routes/  
■   ■■■ auth.py  
■   ■■■ home.py  
■   ■■■ party.py  
■   ■■■ jewelry.py  
■   ■■■ history.py  
■  
■■■ services/  
■   ■■■ gemini_utils.py  
■   ■■■ recommendation_service.py  
■   ■■■ budget_service.py  
■  
■■■ models/  
■   ■■■ user_models.py  
■   ■■■ planner_models.py  
■   ■■■ recommendation_models.py  
■  
■■■ templates/  
■   ■■■ home.html  
■   ■■■ home_planner.html  
■   ■■■ party_planner.html  
■   ■■■ jewelry_planner.html  
■  
■■■ static/  
■   ■■■ css/  
■   ■■■ js/  
■   ■■■ images/
```

Note: This structure is a project-oriented organization based on the uploaded documentation. The source describes routes/, services/, and models/ explicitly; the additional filenames above are a practical way to implement those responsibilities.

4. Component Responsibilities

Component	Responsibility	Examples
main.py	Application startup and registration	FastAPI app, middleware, router inclusion
routes/	HTTP/API endpoints	/generate-home, /generate-party, /generate-jewelry
services/	Business and AI logic	Gemini calls, recommendation generation, budget logic
models/	Structured data definitions	User, planner input, recommendation output
Authentication	Identity/session control	/register, /login, /logout, /token
templates/	Frontend pages	Planner forms and recommendation views
static/	Frontend assets	CSS, JavaScript, images

5. Route Layer

The route layer receives HTTP requests, validates incoming data, calls the appropriate service, and returns a structured response. Routes should not contain large blocks of Gemini prompting or product recommendation logic.

```
# routes/home.py
from fastapi import APIRouter
from services.recommendation_service import get_home_recommendations

router = APIRouter()

@router.post("/generate-home")
async def generate_home(request):
    return await get_home_recommendations(request)
```

Benefit: If the Home Planner changes, its endpoint can be modified without rewriting the Gemini utility or the Party/Jewelry modules.

6. Service Layer & Gemini Integration

The service layer contains the application's reusable intelligence. The uploaded project identifies `gemini_utils.py` as the utility responsible for prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

```
# services/gemini_utils.py
def build_prompt(category, budget, preferences):
    # Build a category-specific Gemini prompt
    ...

def generate_ai_recommendation(prompt, image=None):
    # Send text or text + image to Gemini
    ...

def format_recommendations(response):
    # Convert AI response into application-friendly data
    ...
```

Keeping Gemini communication in one service prevents API-specific code from being duplicated across Home, Party, and Jewelry routes.

7. Model Layer

Models define the structure of information moving through the system. For example, planner inputs can contain budget, category, preferences, quantities, guest count, occasion, and optional image information. Recommendation models can define the expected output fields.

```
# models/planner_models.py
from pydantic import BaseModel
```

```
from typing import Optional

class PlannerRequest(BaseModel):
    budget: float
    category: str
    preferences: Optional[str] = None

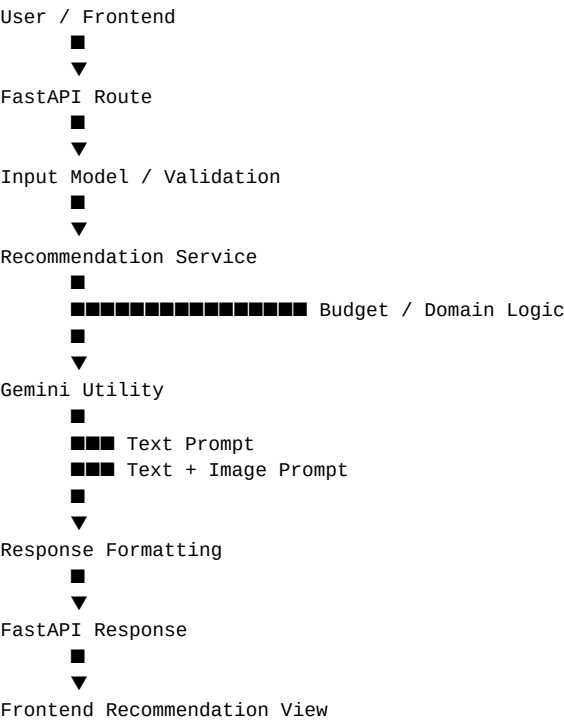
class PartyRequest(BaseModel):
    budget: float
    guests: int
    event_type: str
```

8. Authentication & Session Module

Authentication should remain separate from recommendation generation. The project documentation includes register, login, logout, token, session-info, and session-data functionality. Keeping these responsibilities isolated reduces coupling between user management and AI features.

Endpoint	Responsibility
/register	Create and store a new user account
/login	Authenticate user credentials
/logout	Terminate the current session
/token	Issue an authentication token
/session-info	Return current session metadata
/session-data	Return session-specific personalization data

9. Modular Request Flow



This flow keeps the route focused on communication while the service and utility layers handle the application's recommendation and AI responsibilities.

10. Planner Module Separation

Planner	Input	Service Responsibility
Home	Budget, room types, quantities, preferences	Furniture, decor, lighting recommendations
Party	Budget, guests, event type, venue details	Catering, venue, decoration recommendations
Jewelry	Budget, occasion, style, optional outfit image	Image-aware style matching and jewelry recommendations

The source project explicitly identifies Home, Party, and Jewelry routes and describes their different recommendation responsibilities. [filecite\[turn0file0\[L193-L209\]](#)

11. Modular Structure Validation

ID	Validation	Expected Result
MOD-01	Route imports service successfully	No circular/import errors
MOD-02	Service calls Gemini utility	AI request is isolated from route code
MOD-03	Model validates input	Invalid requests are rejected clearly
MOD-04	Home module changed	Party/Jewelry modules continue working
MOD-05	Gemini utility changed	All planners use the updated shared behavior
MOD-06	Authentication changed	Recommendation service remains independent

12. Final Modular Code Checklist

- ☐ Routes contain endpoint/request-response handling rather than large business-logic blocks.
- ☐ Gemini API calls are centralized in the AI utility/service layer.
- ☐ Recommendation logic is reusable across planner routes.
- ☐ Input/output schemas are defined separately from route logic.
- ☐ Authentication and session functions are isolated.
- ☐ Home, Party, and Jewelry modules are logically separated.
- ☐ Environment variables are used for secrets and API keys.
- ☐ The application can be tested module by module.
- ☐ Changes to one planner do not unnecessarily affect other planners.
- ☐ Startup and router registration remain centralized in the main application.

13. Conclusion

Ensuring a modular code structure gives PocketSmart AI a maintainable foundation for its FastAPI backend and Gemini-powered recommendation engine. The project documentation describes a modular architecture in which routes manage endpoints, services handle recommendation and AI logic, and models define structured data. This separation supports cleaner development, easier testing, and future expansion of the Home, Party, and Jewelry planners.

The project also describes the backend as the bridge between the frontend and Gemini-based AI recommendation engine, with CORS, session handling, and structured input/output models supporting the integration.

Project completion evidence: After implementation, demonstrate the directory structure, run the three planner endpoints, show a successful Gemini response, and record the modular validation checklist.